

NeoChristmasTree

Systembeschreibung und Entwicklungsdokumentation



Hardware: ESP32-S3-WR00M-1 · IP5306 · WS2812B
Platine: TannenbasisV4 (KiCAD 10.0, 2-lagig)
Software: Arduino / ESP-IDF · LittleFS · AsyncWebServer
Gehäuse: 3D-Druck PLA (2-teiliger Einschub-Sockel)

1 Projektübersicht

Der NeoChristmasTree ist ein ESP32-S3-basierter Weihnachtsbaum mit 66 adressierbaren WS2812B-LEDs, integriertem Akkubetrieb, Infrarot-Fernbedienung, WiFi-Webserver und automatischer Helligkeitsanpassung über einen Lichtsensor. Das Projekt richtet sich an Maker, die über grundlegende SMD-Löterfahrung verfügen, und zeigt einige unkonventionelle Lösungsansätze für typische Herausforderungen im Hobby-Bereich.

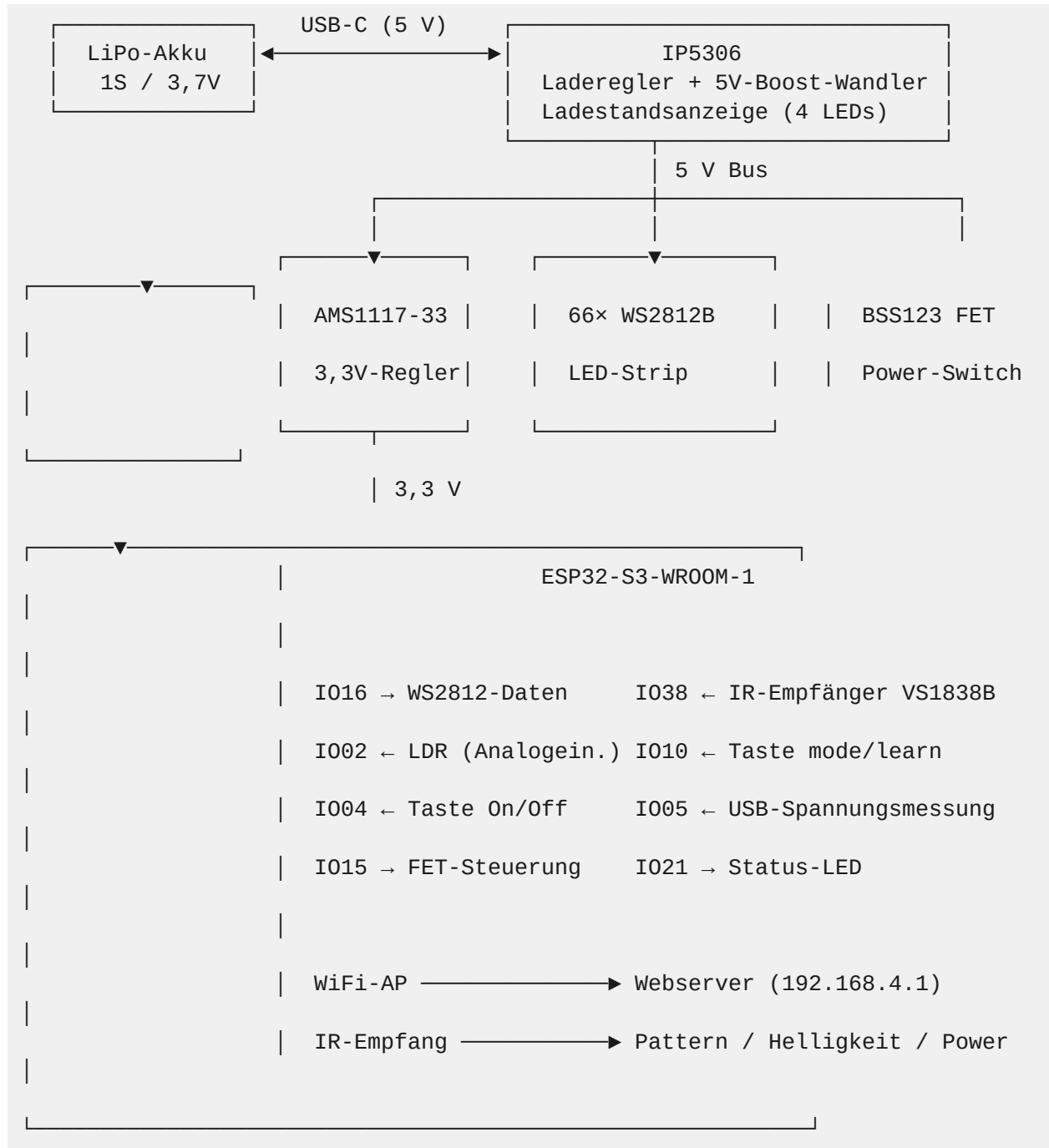
Besondere Merkmale des Projekts:

- Integriertes Akkumanagement mit **IP5306** (Laden und Boost in einem Chip) statt der üblichen zweiteiligen TP4056 + Boost-Converter-Lösung
 - Eigens entwickeltes PCB-Footprint für den IP5306 mit **extra großem GND-Thermal-Pad** (3,65 × 8,55 mm) zum Löten ohne Heißluftstation
 - **USB-C-Stecker** als Ladeeingang — Bestückung unter dem Kameramikroskop
 - Webbasierter **Pattern-Editor** (LittleFS + AsyncWebServer) erreichbar über WiFi-Accesspoint
 - Automatische **LDR-Helligkeitsanpassung** an das Umgebungslicht
 - **2-teiliger 3D-Druck-Sockel** mit Einschub-Mechanismus für die Platine
-

2 Systemarchitektur

2.1 Blockdiagramm

Das folgende Diagramm zeigt den Energiepfad und die Datenflüsse des Systems:



2.2 Betriebsmodi

Modus	Stromversorgung	WiFi	Power-Off
Akkubetrieb	LiPo → IP5306-Boost → 5V	Aus (Stromsparen)	Vollständig (FET-Doppelklick)
USB-Betrieb	USB-C → IP5306 → 5V	AP aktiv	Nur LEDs aus, ESP32 aktiv

3 Hardware-Beschreibung

3.1 Mikrocontroller — ESP32-S3-WROOM-1

Das Herzstück der Schaltung ist das **ESP32-S3-WROOM-1**-Modul von Espressif. Gegenüber dem älteren ESP32 bietet der S3 einen schnelleren Dual-Core-Xtensa-LX7-Prozessor (240 MHz), mehr SRAM (512 KB intern) sowie USB-OTG-Unterstützung. Für dieses Projekt entscheidend ist die reichhaltige Peripherie: ausreichend GPIOs für LEDs, IR, LDR und Taster sowie das integrierte WiFi für den Webserver.

Parameter	Wert
Prozessor	Dual-Core Xtensa LX7, 240 MHz
Flash	4 MB (im Modul, genutzt für LittleFS-Patterns)
RAM	512 KB intern + 8 MB PSRAM (im Modul)
WiFi	802.11 b/g/n, 2,4 GHz
Betriebsspannung	3,3 V
Versorgung	AMS1117-3.3 (aus 5V-Bus des IP5306)

Das Modul enthält bereits alle notwendigen Beschaltungen (Quarz, Filter, Antenne). Auf der Platine sind lediglich die Abblockkondensatoren und die GPIO-Beschaltung nötig.

3.2 Akkumanagement — IP5306 vs. TP4056 + Boost

Die Wahl des Akkumanagement-ICs ist eine der zentralen Designentscheidungen dieses Projekts. In vielen Maker-Projekten wird die Kombination aus **TP4056** (Ladecontroller) und einem separaten **Boost-Converter** (z. B. MT3608 oder SY7208) verwendet. Diese Lösung hat einige Nachteile, die der **IP5306** in einem einzigen Chip behebt.

Warum nicht TP4056 + Boost? Die klassische Kombination benötigt zwei ICs mit je eigenem Induktor, eigener Beschaltung und eigenem Wirkungsgrad-Verlust. Der Ladestrom des TP4056 ist auf 1A begrenzt. Die Gesamteffizienz leidet durch die zwei Wandlerstufen — besonders bei niedrigem Akkustand.

Eigenschaft	TP4056 + MT3608	IP5306 (dieses Projekt)
Integration	2 ICs + 2 Induktoren	1 IC + 1 Induktor
Max. Ladestrom	1,0 A	2,1 A (einstellbar)
Boost-Ausgangsstrom	bis ~2 A (MT3608)	bis 2,4 A
Boost-Effizienz	~80–87 %	~88–93 %
Power-Button-Logik	Extern nötig	Integriert (Doppelklick-Erkennung)
Ladestandsanzeige	Extern (keine oder 1 LED)	4 LEDs direkt angesteuert
Platzbedarf	Groß (2 Footprints)	SOP-8 (8 × 5 mm)
Preis (ca.)	~1,50 € (2 ICs)	~0,80 € (1 IC)

Der IP5306 enthält einen **synchronen Boost-Wandler** mit integrierter Schottky-Diode und einem eigenen Laderegler. Besonders wertvoll für dieses Projekt: Der Chip erkennt Einfach- und Doppelklick auf den Power-Button in Hardware — genau die Logik, die für das Abschalten im Akkubetrieb benötigt wird.

Power-Button-Trick: Im Akkubetrieb schaltet ein Doppelklick auf den Power-Button den IP5306 vollständig ab. Der ESP32 simuliert diesen Doppelklick über einen BSS123-FET auf den Power-Pin — so ist ein vollständiges Software-Power-Off möglich, ohne dass der Nutzer den Taster zweimal drücken muss.

Die vier Ladestand-LEDs (D1–D4, bestückt als SMD-LEDs 1210) zeigen in 25-%-Schritten den Akkustand an. Der IP5306 treibt diese LEDs direkt ohne externe Logik.

3.3 3,3V-Versorgung — AMS1117-3.3

Aus dem 5V-Bus des IP5306 erzeugt ein **AMS1117-3.3** im SOT-223-Gehäuse die 3,3V für den ESP32-S3. Der AMS1117 ist ein LDO-Regler (Low-Dropout) mit bis zu 800 mA Ausgangsstrom — ausreichend für den ESP32 inklusive WiFi-Betrieb. Die Dropout-Spannung von ca. 1,2V ist bei einem 5V-Eingang unkritisch.

Parallel zum AMS1117 liegen Abblockkondensatoren (100nF keramisch + 100µF elektrolytisch) um Lastsprünge durch WiFi-Bursts und LED-Datenübertragung zu glätten.

3.4 Ladeeingang — USB-C-Stecker (Lötbesonderheit!)

Als Ladeeingang dient ein **USB-C-Plug** in einem eigens erstellten KiCAD-Footprint. USB-C bietet gegenüber Micro-USB den Vorteil der Steckerrichtungsfreiheit und höherer mechanischer Belastbarkeit — relevant für ein Gerät, das jede Weihnachtssaison aus der Schublade geholt und angeschlossen wird.

Löthinweis — Kameramikroskop empfohlen: Der USB-C-Connector hat Signalpins (CC1/CC2, D+/D−) mit einer Breite von nur **0,3 mm** und einem Pitch von 0,5 mm. Die GND- und VBUS-Pads messen 0,6 mm. Ein Kameramikroskop (oder zumindest eine starke Lupe mit guter Beleuchtung) ist beim Bestücken und Verlöten dringend empfohlen, um Brücken zwischen den Pins zu erkennen und zu beseitigen. Flussmittel-Gel und eine feine Lötspitze (max. 0,3 mm) sind Pflicht.

Mechanisch hält der Connector über vier **Through-Hole-Lötösen** (ovale Pads 1,0 × 1,8 mm und 1,0 × 2,1 mm), die auch von Hand gut verlötbar sind und die nötige Zugentlastung bieten.

3.5 Infrarot-Empfänger — VS1838B

Der **VS1838B** (Footprint: TSDP341xx) ist ein kompakter 38-kHz-IR-Empfänger im 3-Pin-Gehäuse. Er demoduliert das Trägersignal und liefert direkt das NEC-Protokoll-Signal an GPIO38 des ESP32. Die Library **IRremote** übernimmt die Dekodierung.

Besonderheit im Code: ``strip.show()`` wird nur aufgerufen, wenn ``IrReceiver.isIdle()`` true ist, um elektromagnetische Interferenzen zwischen WS2812-Datenleitung und IR-Empfänger zu vermeiden.

3.6 Umgebungslicht — LDR-Spannungsteiler

An GPIO2 liegt ein Spannungsteiler aus **LDR** (gegen VCC) und **100 kΩ** (gegen GND). Bei hellem Umgebungslicht sinkt der LDR-Widerstand → höhere Spannung an GPIO2 → höherer ADC-Wert → hellere LEDs. Bei Dunkelheit kehrt sich das um.

Der ESP32 liest den ADC-Wert alle 500 ms (konfigurierbar über `AUTO\_BR\_INTERVAL`) und mappt ihn linear auf den Helligkeitsbereich `[AUTO\_BR\_MIN, gBrightness]`. Dabei wirkt der per Fernbedienung eingestellte Wert `gBrightness` als Obergrenze — die automatische Helligkeit kann nie höher gehen als die manuelle Einstellung.

```
#define LDR_DARK          200    // ADC-Wert bei Dunkelheit → kalibrieren!
#define LDR_BRIGHT       3500   // ADC-Wert bei hellem Licht → kalibrieren!
#define AUTO_BR_MIN       3      // LED-Helligkeit bei Dunkelheit
#define AUTO_BR_MAX       255    // LED-Helligkeit bei vollem Licht
#define AUTO_BR_INTERVAL  500    // Messintervall in ms
```

3.7 Power-Management — BSS123 N-Kanal-FET

Ein **BSS123** (SOT-23, N-Kanal-MOSFET, 170 mA, 100 V) verbindet GPIO15 des ESP32 mit dem Power-Button-Pin des IP5306. Durch gezielte Pulse auf GPIO15 simuliert der ESP32 den für den IP5306 erforderlichen Doppelklick zum Abschalten. Im USB-Betrieb wird dieser Pfad nicht genutzt (IP5306 kann über USB nicht vollständig abschalten).

4 Platinen-Design (TannenbasisV4)

4.1 Übersicht

Parameter	Wert
Design-Tool	KiCAD 10.0
Lagen	2 (F.Cu, B.Cu)
Abmessungen	ca. 80 × 60 mm (Schätzwert aus Bauteilkoordinaten)
Oberfläche	HASL oder ENIG empfohlen
Bestückung	Überwiegend SMD 0805, wenige THT-Bauteile

4.2 IP5306 — Das besondere Lötpad

Der IP5306 im SOP-8-Gehäuse hat acht Standard-Gullwing-Pads ($2,85 \times 0,7$ mm) und ein thermisches Exposed-Pad (Pad 9, GND) auf der Unterseite. Dieses Thermal-Pad ist für die Wärmeableitung essenziell — und genau hier liegt die konstruktive Besonderheit dieses Designs.

Lötpad-Größe im Vergleich: Das Thermal-Pad des IP5306 ist im Standard-Datenblatt mit ca. $3,3 \times 3,3$ mm angegeben. Im Footprint dieses Projekts wurde ein **kombiniertes Pad von $3,65 \times 8,55$ mm** definiert (inklusive Lötbrücken-Flächen). Zusätzlich gibt es drei Thermal-Via-Löcher ($\varnothing 0,5$ mm) auf beiden Seiten des Chips, die das Pad für einen StandardlötKolben zugänglich machen.

Das Ergebnis: Wer keinen Heißluftlötter besitzt, kann den IP5306 mit einem **100-Watt-LötKolben** und ausreichend Flussmittel von der Seite über die Thermal-Vias verlöten. Die Vias leiten Wärme und Lot unter den Chip.

4.3 Schaltplan und Layout

Die folgenden Dokumente liegen dem Projekt bei:

- `docs/schematic.pdf` — vollständiger Schaltplan (exportiert aus KiCAD)
- `docs/pcb\_front.pdf` — PCB-Vorderseite (F.Cu + Silkscreen + Outline)
- `docs/pcb\_back.pdf` — PCB-Rückseite (B.Cu + Silkscreen + Outline)
- `TannenbasisV4-produce/` — fertige Gerber-Dateien für die Platinenfertigung

Die Gerber-Dateien sind direkt bei Standard-PCB-Fertigern (z. B. JLCPCB, PCBWay) bestellbar.

5 3D-Druck-Sockel

5.1 Konzept

Der Sockel ist als **zweiteiliger Einschub** konstruiert. Die Platine wird in den Hauptkörper eingeschoben und durch ein Endstück gesichert — keine Schrauben, kein Kleber. Das ermöglicht einfachen Zugang zur Platine für Wartung oder Umprogrammierung.

Datei	Beschreibung
`XmasTree-Base3-2-main.stl`	Hauptkörper mit Einschubführung für die Platine
`XmasTree-Base3-2-end.stl`	Abschlusskappe, sichert die Platine im Sockel

5.2 Druckempfehlungen

- Material: **PLA** (ausreichend für Innenbereich)
- Schichthöhe: **0,2 mm** (Standardqualität)
- Infill: **20 %** (Gyroid oder Grid)
- Supports: nicht erforderlich (Teile sind support-frei konstruiert)
- Perimeter: **3 Wände** für ausreichende Stabilität der Einschubführung

6 Software-Architektur

6.1 Übersicht der Module

Der Quellcode ist in mehrere `.ino`-Dateien aufgeteilt, die die Arduino-IDE gemeinsam kompiliert:

Datei	Funktion
`NeoChristmasTreeV2.ino`	Hauptdatei: Setup, Loop, Auto-Brightness, Power-Off-Logik
`pattering.ino`	Pattern-Renderer: liest aus PatternEngine, setzt LEDs
`IRhandling.ino`	IR-Empfang, IR-Lernmodus (12 Codes × 3-fache Bestätigung)
`filesystem.ino`	LittleFS: Pattern-Index laden, Pattern-JSON lesen/schreiben
`webserver.ino`	WiFi-AP + AsyncWebServer + REST-API für den Pattern-Editor
`myPrefHandling.ino`	NVS-Preferences: IR-Codes, Pattern-ID,

	Helligkeit persistent
`serialprintf.ino`	sprintf-Wrapper für komfortables Serial-Debugging
`PatternEngine.h`	Datenstruktur für geladene Patterns (Frames, Farben, Timing)
`AllPattern.h`	Definitionen eingebauter Fallback-Patterns
`IRcodes.h`	Default-IR-Codes (Golden Power Ningbo Fernbedienung)

6.2 Pattern-System (LittleFS + JSON)

Lichtmuster werden als **JSON-Dateien** im LittleFS-Dateisystem des ESP32-Flash gespeichert. Ein zentraler Index (`/patterns/\_index.json`) listet alle verfügbaren Patterns mit ID und Anzeigename.

```
// Verzeichnisstruktur im Flash:
/patterns/
  _index.json          ← Liste aller Patterns
  [{"id":"rainbow","name":"Regenbogen"}, ...]
  rainbow.json         ← Pattern-Daten (Frames, Farben, Timing)
  snowfall.json
  ...
/www/
  index.html           ← Pattern-Editor (Single-Page-App)
```

Das aktive Pattern wird über eine **String-ID** (z. B. "rainbow") referenziert — nicht über einen Integer-Index. Das macht Patterns unabhängig von ihrer Reihenfolge und ermöglicht das Hinzufügen/Entfernen ohne Neukompilierung der Firmware.

6.3 WiFi-Accesspoint und Web-Editor

Im USB-Betrieb öffnet der ESP32 automatisch einen **WiFi-Accesspoint** (SSID: ChristmasTree, kein Passwort). Der AsyncWebServer liefert die Single-Page-App (HTML/JavaScript) und stellt folgende REST-Endpunkte bereit:

Endpunkt	Methode	Funktion
`/`	GET	Liefert index.html (Pattern-Editor-UI)

<code>/api/patterns`</code>	GET	Gibt _index.json zurück (Patternliste)
<code>/api/pattern?id=X`</code>	GET	Gibt Pattern-JSON für ID X zurück
<code>/api/pattern`</code>	POST	Speichert Pattern-JSON (id im Body)
<code>/api/activate`</code>	POST	Aktiviert Pattern direkt (ohne Speichern)
<code>/api/brightness`</code>	POST	Setzt Helligkeit live

6.4 IR-Fernbedienung

Der IR-Empfänger wird über die **IRremote-Library** ausgelesen. Zwölf IR-Codes sind konfigurierbar (Patterns 1–8, OFF, ON, Dunkler, Heller) und werden im NVS-Flash gespeichert. Der Lernmodus (Taste ``mode/learn`` beim Einschalten gehalten) erfasst neue Codes mit **dreifacher Bestätigung**, um instabile China-Fernbedienungen zu tolerieren.

6.5 Automatische Helligkeit (LDR)

Die Auto-Brightness-Funktion liest den LDR-Wert alle ``AUTO_BR_INTERVAL`` Millisekunden und mappt ihn über ``constrain()`` und ``map()`` auf den Helligkeitsbereich:

```
void updateAutoBrightness() {
  if (millis() - gLastLdrCheck < AUTO_BR_INTERVAL) return;
  gLastLdrCheck = millis();
  int raw      = analogRead(ROOMLIGHT);
  int clamped  = constrain(raw, LDR_DARK, LDR_BRIGHT);
  int autoBr   = map(clamped, LDR_DARK, LDR_BRIGHT, AUTO_BR_MIN,
AUTO_BR_MAX);
  autoBr      = constrain(autoBr, AUTO_BR_MIN, gBrightness); // IR-Wert als
Deckel
  strip.setBrightness(autoBr);
}
```

6.6 Power-Management im Code

Die Firmware unterscheidet USB- und Akkubetrieb anhand des analogen Spannungswerts an `USB5VSENSE` (GPIO5). Liegt dieser unter 1000 (ADC-Rohwert), wird Akkubetrieb angenommen:

- **USB:** WiFi-AP starten, echter Power-Off nicht möglich → `offpattern` anzeigen
- **Akku:** WiFi aus, bei Ausschalt-Befehl FET-Doppelklick ausführen (`powerOff()`)

7 Stückliste (BOM)

Automatisch extrahiert aus `TannenbasisV4.kicad\_sch` (KiCAD 10.0). Alle SMD-Widerstände und -Kondensatoren sind im Format **0805** ausgeführt (handlöttauglich).

Elektrolytkondensatoren sind bedrahtet (Radial).

Stk.	Referenz(en)	Wert / Typ	Footprint
3	C1, C2, C13	10µF	C_0805_2012Metric
1	C14	100n	C_0805_2012Metric
2	C3, C8	100nF	C_0805_2012Metric
2	C4, C12	100µF	CP_Radial_D6.3mm_P2.50mm
6	C5, C6, C7, C9, C10, C11	22µF	C_1210_3225Metric_Pad1.33x2.70mm
1	D1	25%	LED_1210_3225Metric_Pad1.42x2.65mm
1	D2	50%	LED_1210_3225Metric_Pad1.42x2.65mm
1	D3	75%	LED_1210_3225Metric_Pad1.42x2.65mm
1	D4	100%	LED_1210_3225Metric_Pad1.42x2.65mm
1	D6	LED	LED_1210_3225Metric_Pad1.42x2.65mm
1	J1	Battery	PinHeader_1x02_P2.54mm_Vertical
2	J10, J11	LED2in	solderconnect-1
1	J12	USB_B_Micro	USB_Micro-B_Pins
2	J13, J14	mount	solderconnect-1

1	J15	I2Cb	PinHeader_1x04_P2.54mm_Vertical
1	J16	I2Ca	PinHeader_1x04_P2.54mm_Vertical
1	J18	LDR	PinHeader_1x02_P2.54mm_Vertical
1	J19	Developer-IO	PinHeader_2x14_P2.54mm_Vertical
2	J2, J3	LEDvdd	solderconnect-1
1	J20	I2Cc	PinHeader_1x04_P2.54mm_Vertical
2	J4, J5	LEDvss	solderconnect-1
2	J6, J7	LEDdata	solderconnect-1
2	J8, J9	LED1out	solderconnect-1
1	L1	1 μ H	MyPowerCoil
1	P2	USB-C-Plug-My	MyUSB-C
1	Q1	BSS123	SOT-23
4	R1, R2, R7, R8	1 Ω	R_0805_2012Metric
3	R10, R12, R14	100 Ω	R_0805_2012Metric
1	R11	220 Ω	R_0805_2012Metric
1	R17	1k	R_0805_2012Metric
2	R3, R4	5k1	R_0805_2012Metric
3	R5, R16, R19	100k	R_0805_2012Metric
5	R6, R9, R13, R15, R18	10k	R_0805_2012Metric
1	SW1	On/Off	SW_DIP_SPSTx01_Slide_6.7x4.1mm_W8.61mm_P2.54mm_Low Profile
1	SW2	Reset	SW_DIP_SPSTx01_Slide_6.7x4.1mm_W8.61mm_P2.54mm_Low Profile
1	SW3	mode/learn	SW_DIP_SPSTx01_Slide_6.7x4.1mm_W8.61mm_P2.54mm_Low

			Profile
1	SW4	Boot/Flash	SW_DIP_SPSTx01_Slide_6.7x4.1mm_W8.61mm_P2.54mm_Low Profile
1	U1	IP5306	IP5306
1	U2	AMS1117-3.3	SOT-223-3_TabPin2
1	U3	USBLC6-2SC6	SOT-23-6
1	U4	VS1838B	Vishay_MOLD-3Pin
1	U5	ESP32-S3-WROOM-1	ESP32-S3-WROOM-1

Gesamt: 69 Bauteile in 42 verschiedenen Typen.

8 Inbetriebnahme

8.1 Firmware flashen

1. Arduino-IDE installieren und das **ESP32-Boardpaket** (Espressif, Version ≥ 2.0) einrichten.
2. Folgende Libraries installieren: **IRremote**, **Adafruit NeoPixel**, **ESPAsyncWebServer**, **ArduinoJson**, **LittleFS**.
3. Den ESP32 über den **USB-Micro-B-Connector (J12)** mit dem Computer verbinden.
4. Das Sketch `NeoChristmasTreeV2.ino` öffnen und auf den ESP32 hochladen.
5. Anschließend die **LittleFS-Daten** hochladen (Arduino-IDE → Werkzeuge → ESP32 LittleFS Data Upload). Enthält Pattern-JSON und den Web-Editor.

8.2 LDR kalibrieren

Die Rohwerte des LDR sind je nach verbautem Exemplar und Lichtbedingungen unterschiedlich. Kalibrierung:

1. Baum einschalten und Serial Monitor (115200 Baud) öffnen.
2. Den Baum in **vollständiger Dunkelheit** aufstellen → `ROOMLIGHT`-Wert notieren → als `LDR\_DARK` eintragen.

3. Den Baum bei **normaler Raumbeleuchtung** aufstellen → Wert notieren → als `LDR\_BRIGHT` eintragen.
4. Firmware neu flashen.

8.3 IR-Fernbedienung anlernen

Falls eine andere als die mitgelieferte Fernbedienung (Golden Power Ningbo) verwendet werden soll: Taste `mode/learn` beim Einschalten gedrückt halten → Lernmodus. 12 Tasten nacheinander **dreimal** drücken. Details: siehe Bedienungsanleitung.